

2026 年 6 月 19 日 Version 1.1

ハッカソン ～計画書と設計書～

チーム 9

24G1098 : 手代木 巧
24G1052 久家 力孔
24G1145 渡邊 乃斗
25G1010 家田 祐希
25G1043 木下 遼介
25G1046 櫛濱 和輝

目次

第 1 章 プロジェクトの計画書

1.1 プロジェクトの概要

本プロジェクトでは、5 台の Arduino を用いたオーケストラシステムの作成を行う。演奏楽曲は「かえるのうた」とし、Arduino のうち一台を指揮用、4 台を演奏用として使用し、楽器構成はピアノ・フルート・トランペット・ドラムとした。Arduino 間の同期方法は赤外線通信を使用し、1 台の指揮側 Arduino が赤外線信号を通じて 5 台の演奏側 Arduino へ同期信号を送信し、輪唱形式で演奏する。また、エンターテイメント性として演奏側にはキャラクターディスプレイを接続し、演奏曲の歌詞と歌うかえるのアニメーションを再生する。

本プロジェクトでは、1 週目から 5 週目までを計画段階、6 週目から 9 週目までを実装段階とし、10 週目に包括・結合を予定している。

1.1.1 プロジェクトの題目

プロジェクト題目は、「赤外線通信で歌うカエル」とする。

1.1.2 プロジェクトの目的

近年、IoT 技術の普及により、複数のデバイスがネットワークを介してリアルタイムに連携するシステムが様々な分野で実用化されている。製造業における機械の協調制御、医療分野におけるセンサネットワーク、スマートホームにおける機器連携など、複数の自律した機器を同期させることが現代の組み込みシステムでは求められている[?]。本プロジェクトで対応する楽器間の同期というテーマにおいても、楽器が互いにズれることなく音を重ねるためには、リアルタイムな状態伝達が不可欠である。

既存の電子合奏システムの多くは Wi-Fi や Bluetooth といったネットワーク通信に依存しており、通信インフラの整備や環境への依存という制約が残る[?]。また、MIDI のような高精度な同期規格も存在するが、専用インターフェースや対応機器が必要であり、小規模な組み込みシステムへの導入にはコストと複雑さが伴う。そこで、本プロ

ジェクトでは、通信インフラや専用規格に依存せず、複数の機器をリアルタイムで同期させることを目的とする。

1.1.3 既存の技術とプロジェクトの独自性

1.1.3.1 既存の技術とプロジェクトの独自性

電子合奏・楽器同期に関連する既存技術として、MIDI や Wi-Fi・Bluetooth を用いたシステムが挙げられる。MIDI は音の高さやタイミング、強さなど精密な情報を伝えることができるが、専用インターフェースや対応ライブラリが必要であり、小規模な組み込みシステムへの導入にはコストと複雑さが伴う。Wi-Fi や Bluetooth を用いた合奏システムは遅延補正技術も発達しているが、通信インフラの品質に動作が左右されるため、実験室環境では電波干渉や接続不安定のリスクが生じる [?].

本プロジェクトにおける同期では、各演奏側 Arduino に対して必要最小限の演奏参加情報のみを伝達することを目的としている。そこで本プロジェクトでは、音楽データ通信ではなく「演奏参加タイミングの同期」に機能を限定し、赤外線通信を用いた独自の同期システムを構築する。具体的には、指揮側 Arduino が演奏中の楽器集合を表す mask データを赤外線で全子機へ一斉送信し、各演奏側 Arduino はその mask のうち自身の担当ビットのみを参照して演奏可否を判定する。さらに、各楽器について待機状態 (OFF)、開始予約状態 (QUEUED)、演奏中状態 (ACTIVE) を管理し、演奏者が任意のタイミングで入力した開始要求を次の小節頭で反映することで、人間の入力タイミングのばらつきを吸収しつつ輪唱を同期させる。

また、赤外線通信はネットワーク環境を必要とせず、LED と受光モジュールのみで構成可能であるため、小規模な組み込みシステムでも低コストで実装できる [?]. このように、必要最小限の同期情報に機能を限定しつつ、mask による一斉送信と QUEUED / ACTIVE による開始予約制御を組み合わせることで、シンプルな構成とリアルタイム性を両立する点が本プロジェクトの独自性である。

1.2 スコープ・マネジメント

1.2.1 成果物

本プロジェクトの成果物は、赤外線通信を用いた指揮側 Arduino1 台と演奏側 Arduino 4 台で構成されるオーケストラシステムである。指揮側 Arduino は 1 つの赤外線 LED を用いて、演奏中の楽器集合を表すマスクデータを全演奏側へ一斉送信する。演奏側 Arduino はそれぞれ 1 基の赤外線受光機でこのマスクデータを受信し、自身の担当楽器に対応するビットが有効である場合にのみ、子機内に保持した譜面配列を 1 コマ進めて演奏する。また、指揮側 Arduino は各楽器について待機状態（OFF）、開始予約状態（QUEUED）、演奏中状態（ACTIVE）を管理し、演奏者が任意のタイミングで押した楽器スイッチ入力を予約として受け付け、小節の先頭で演奏開始させる。各演奏側 Arduino はシリアル通信によって Processing と接続し、Processing が倍音構造・ADSR 制御・エフェクトを含む音色合成と音の再生を担う。加えて、LCD キャラクターディスプレイモジュールを接続し、カエルの口パクアニメーションと歌詞の表示を行う。課題曲は「かえるのうた」の輪唱とする。

1.2.2 プロジェクトの対象範囲と非対象範囲

本成果物の対象範囲は、赤外線による同期制御、Arduino での音符情報生成、Processing での音声出力、LCD による視覚演出、および可変テンポへの対応とする。一方、Wi-Fi や Bluetooth 等の無線ネットワーク通信や MIDI 等による有線通信は対象範囲外とする。

1.2.3 機能分解構造 FBS

本プロジェクトにおける成果物の FBS を図??に記す。オーケストラシステムの機能は、大きく分けて同期制御、楽曲制御、エンターテイメントの 3 つである。各機能をさらに細分化すると、同期制御は信号の送受信、楽曲制御は音符生成・演奏タイミング・音声出力、エンターテイメントは LCD 表示にそれぞれ分解される。

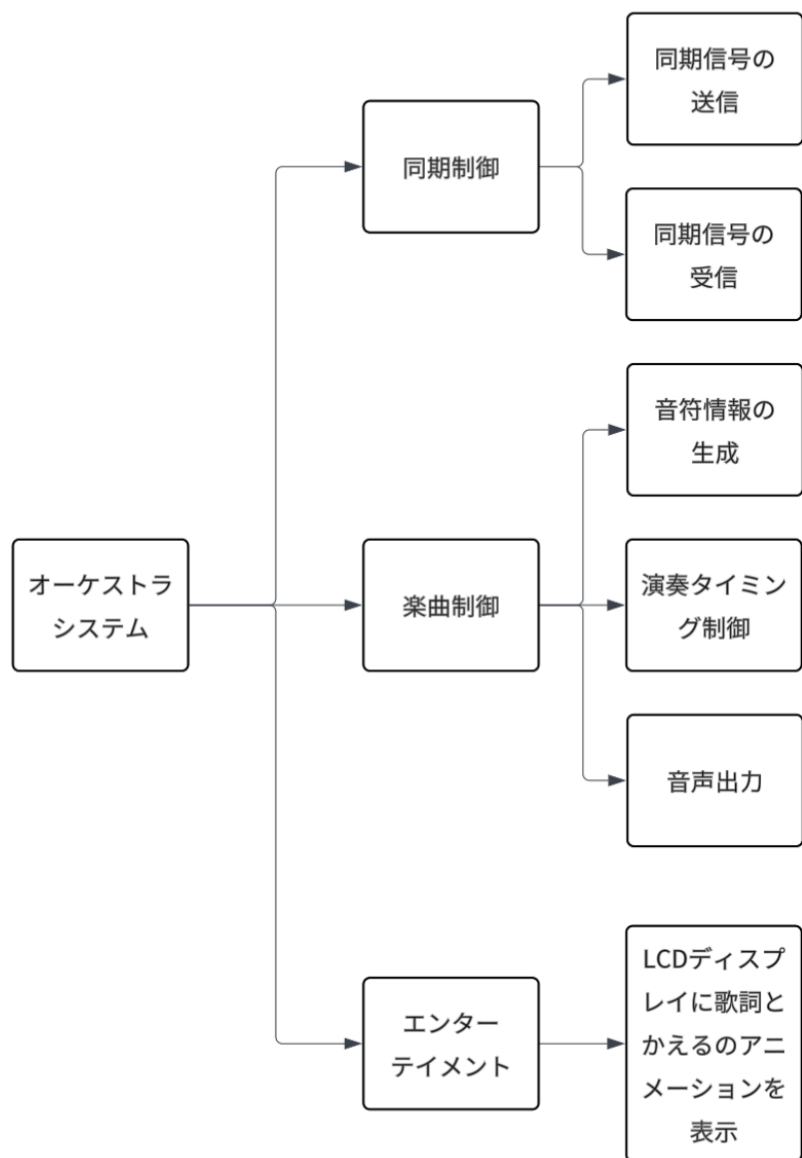


図 1.1 作成システムの FBS 図

1.2.4 製品分解構造 PBS

本プロジェクトにおける成果物の PBS を図??に記す。PBS では、成果物を Processing, 指揮側 Arduino, 演奏側 Arduino の 3 つに分けた。Processing は音色・エフェクト付与関数, 音声再生関数, 演奏側との通信関数から構成される。指揮側 Arduino は演奏側との同期プロトコルおよび同期用ハードウェアから構成され、同期プロトコルは、楽器音 on/off 切り替え用赤外線管理プログラムと楽譜進行用赤外線管理プログラムに、同期用ハードウェアは赤外線ライト 1・2 にそれぞれ分解される。演奏側 Arduino は楽譜情報, Processing との通信関数, 指揮側との同期プロトコル, 同期用ハードウェア, LCD 表示関数から構成される。楽譜情報は音符データおよび音符配列に、指揮側との同期プロトコルは楽譜進行プログラムおよび楽器音管理プログラムに、同期用ハードウェアは IR センサ 1・2 に、LCD 表示関数はかえるのアニメーション表示プログラムおよび歌詞表示用プログラムにそれぞれ分解される。

1.2.5 FBS と PBS の対応関係

本プロジェクトで作成するシステムの FBS と PBS は、表??のように対応している。この表は、図?? 中のオーケストラシステムの下に位置するノード群を第 1 層、その下に位置する群を第 2 層、それに対応する PBS のノードを対応する PBS 成果物として示している。

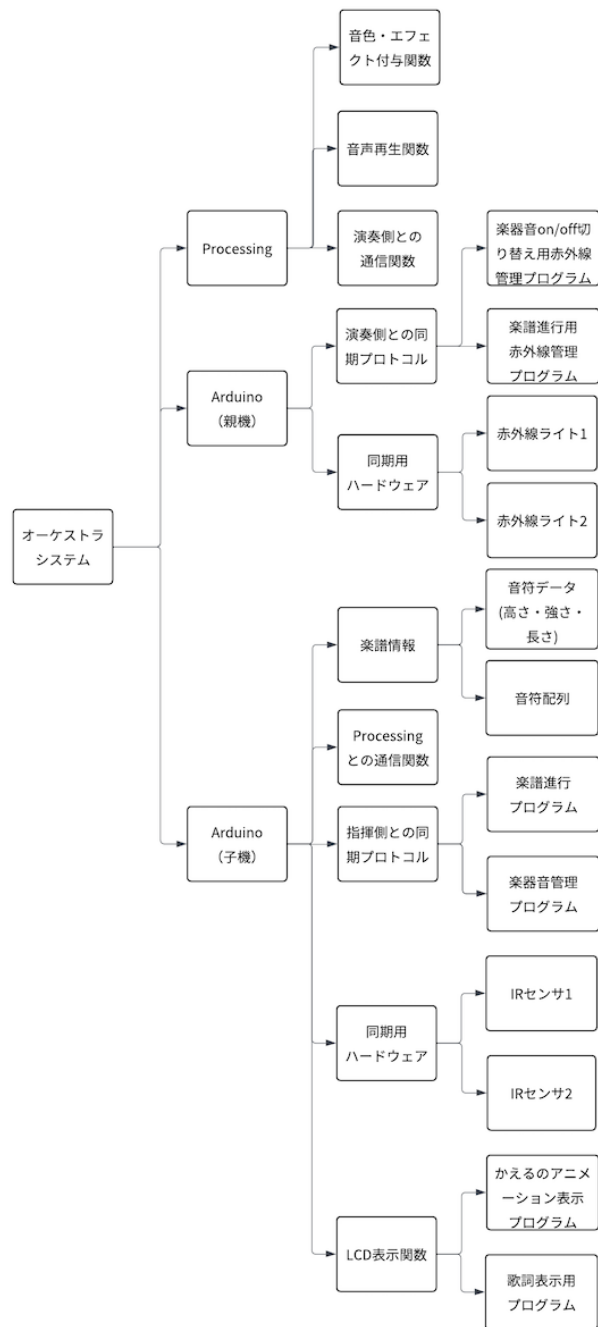


図 1.2 作成システムの PBS 図

表 1.1 FBS と PBS の対応

FBS(第 1 層)	FBS (第 2 層)	対応する PBS 成果物
同期制御	同期信号の送信	指揮側 Arduino / 演奏側との同期プロトコル / 演奏状態管理プログラム, 予約開始制御プログラム, マスク生成プログラム, 赤外線送信管理プログラム. 指揮側 Arduino / 同期用ハードウェア / 赤外線 LED, タクトスイッチ群.
音符情報の生成	同期信号の受信	演奏側 Arduino / 指揮側との同期プロトコル / マスク受信プログラム, 楽譜進行プログラム, 発音判定プログラム. 演奏側 Arduino / 同期用ハードウェア / IR センサ.
	演奏側 Arduino / 楽譜情報 / 音符データ, 音符配列.	楽曲制御
	演奏タイミング 制御	指揮側 Arduino / 演奏側との同期プロトコル / 演奏状態管理プログラム, 予約開始制御プログラム. 演奏側 Arduino / 楽譜情報 / 音符配列.
LCD ディスプレイに歌詞と かえるのアニメーションを	音声出力	Processing / 音色・エフェクト付与関数, 音声再生関数, 演奏側との通信関数. 演奏側 Arduino / Processing との通信関数. エンターテイメント
	演奏側 Arduino / LCD 表示関数 / かえるのアニメーションを	

1.3 作業計画

1.3.1 作業分解構造 WBS と管理番号及び作業分担

本プロジェクトにおける作業分解構造 WBS と管理番号及び作業分担を図??に示す。作成物は, Processing, Arduino(子機), Arduino(親機), その他に分け管理番号を付与した。Processing では, 主に各楽器の音源データ作成および Arduino との通信用関数を担当する。Arduino 子機では, 各楽器の制御や楽曲の進行を行う関数を作成する。また, Arduino 親機では, 赤外線信号の発信などの全体の統括を行うプログラムの開発を行う。その他では回路作成や LED マトリクス表示などの周辺ハードウェア関連作業を行う。

第1層	第2層：要素成果物	第3層：活動タスク	担当者
実装フェーズ	100：processing	110:ピアノ音の作成	柳濱和輝
		120:arduinoとの通信用関数（ピアノ）	柳濱和輝
		130:フルート音の作成	冢田祐希
		140:arduinoとの通信用関数（フルート）	冢田祐希
		150:トランペット音の作成	木下遼介
		160:arduinoとの通信用関数（トランペット）	木下遼介
		170:ドラム音の作成(キック)	久冢力孔
		171:ドラム音の作成（シンバル）	久冢力孔
		180:arduinoとの通信用関数（ドラム）	久冢力孔
	200：Arduino（子機）	210:ピアノのプログラム	柳濱和輝
		220:フルートのプログラム	冢田祐希
		230:トランペットのプログラム	木下遼介
		240:楽譜の設定	渡邊乃斗
		250:楽譜進行プログラム	渡邊乃斗
	300：Arduino（親機）	310：赤外線管理プログラム	渡邊乃斗
		320：テンポ管理プログラム	渡邊乃斗
	400：その他	410:回路作成	手代木巧
		420:LEDマトリクス作成	手代木巧
	500：包括・合成	510:結合テスト	全員

図 1.3 WBS と役割分担

1.3.2 アローダイアグラムとクリティカルパス

本プロジェクトにおける活動タスクの依存関係をアローダイアグラムとして図??に示す。各活動グループの完了を 1 つのノードとして表現し、タスク間の依存関係を矢印で示している。Processing（100 番台）は Arduino 子機（200 番台）との通信用関数を含むため、Processing 前半（110～171）の完了を受けて後半（120～180）が進む構成とした。Arduino 親機（300 番台）、その他（400 番台）はいずれも第 8 週末までに完了し、第 10 週の結合テスト（510）に合流する。

クリティカルパスは, Arduino 子機 (210~250) から結合テスト (510) への経路であり, 総期間は第 6 週から第 10 週末までの 5 週間となる. Arduino 子機の実装は最も多くのタスクを含み, Processing 完了への依存関係も持つため, このパス上での遅延がプロジェクト全体の遅延に直結する. したがって, Arduino 子機の開発進捗を重点的に監視し, 遅延の兆候が見られた場合は速やかに役割を再配分する.

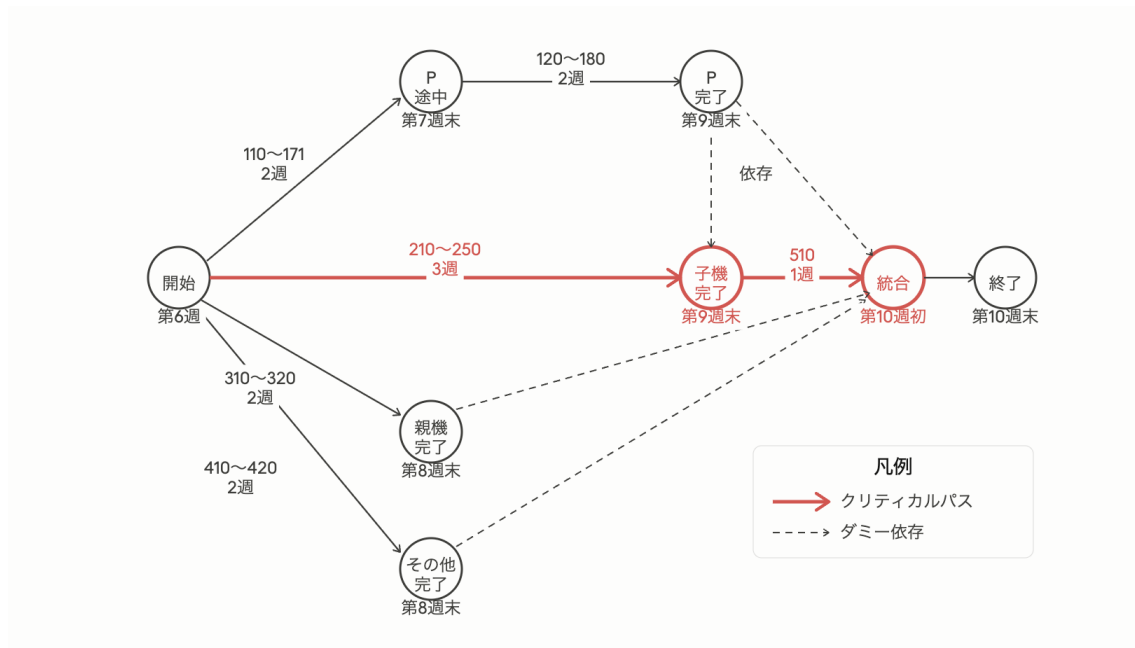


図 1.4 アローチャートとクリティカルパス

1.3.3 ガントチャート

本プロジェクトのスケジュールをガントチャートとして図??に示す. 各期間はアローダイアグラムをもとに決定している. また, 最終週を結合テスト (510) の期間として確保し, 各タスクへの個別バッファは設けずプロジェクト全体で吸収する構成とした.

第3層：活動タスク		担当者	6週目	7週目	8週目	9週目	10週目
110:	ピアノ音の作成	柳濱和輝					
120:	arduinoとの通信用関数（ピアノ）	柳濱和輝					
130:	フルート音の作成	冢田祐希					
140:	arduinoとの通信用関数（フルート）	冢田祐希					
150:	トランペット音の作成	木下遼介					
160:	arduinoとの通信用関数（トランペット）	木下遼介					
170:	ドラム音の作成(キック)	久冢力孔					
171:	ドラム音の作成（シンバル）	久冢力孔					
180:	arduinoとの通信用関数（ドラム）	久冢力孔					
210:	ピアノのプログラム	柳濱和輝					
220:	フルートのプログラム	冢田祐希					
230:	トランペットのプログラム	木下遼介					
240:	楽譜の設定	渡邊乃斗					
250:	楽譜進行プログラム	渡邊乃斗					
310:	赤外線管理プログラム	渡邊乃斗					
320:	テンポ管理プログラム	渡邊乃斗					
410:	回路作成	手代木巧					
420:	LEDマトリクス作成	手代木巧					
510:	結合テスト	全員					

図 1.5 ガントチャート

1.4 検証と妥当性確認: V&V 計画

1.4.1 プロジェクトの成果物に対する有効指標 MOE

本プロジェクトの有効指標 MOE を表??に示す。MOE は成果物の有効性を判断するための指標であり、本プロジェクトでは実際にオーケストラの演奏を聴く人をステークホルダとする。

1.4.2 有効指標 MOE に対応する性能指標 MOP

各 MOE を達成するために必要なシステムの性能指標 MOP を表??に示す。先行研究によると、複数の楽器を用いた演奏において聴衆が演奏のズレを知覚し始める閾値は 20～30 ms 程度とされているため [?, ?], 30ms を演奏タイミング誤差の上限値の根拠とする。また, Arrighi et al. (2006) では, 楽器演奏における視聴覚の同時性を得るためには、音と視覚間遅延が、約 30～100 ms である必要があることを報告されている [?]. したがって、本システムの LCD 表示の同期許容範囲の上限値は 100 ms とする。

1.4.3 システムテストで評価する性能指標 MOP

システムテストでは、全 PBS 成果物を統合した状態で、MOE1～MOE3 の達成を総合的に評価する。評価する MOP を表??に示す。

表 1.2 有効指標 MOE

No.	MOE	評価基準
MOE1	違和感なく聞けることができる	「かえるのうた」の輪唱が、聴衆が聴いてズレを感じることなく演奏できている。また、音が意図せず途切れることなく最後まで演奏できる
MOE2	可変テンポに対応している	指揮側 Arduino の信号頻度を变化させることで、演奏テンポをリアルタイムに変更できる
MOE3	楽器音らしい音色である	Processing で合成した音が楽器音として聴衆に認識される
MOE4	歌詞・アニメーションが演奏に対応している	演奏進行に合わせて LCD の歌詞表示および口パクアニメーションが同期して動作し、違和感がないこと

1.4.4 結合テストで評価する性能指標 MOP

結合テストでは、指揮側 Arduino と演奏側 Arduino、および演奏側 Arduino と Processing をそれぞれ接続した状態で、サブシステム間の連携を評価する。評価する MOP を表??に示す。

1.4.5 単体テストで評価する性能指標 MOP

単体テストでは、各成果物を単独で動作させ、個別の機能が正しく動作することを確認する。評価する MOP を表??に示す。

1.4.6 プロジェクト中に監視する技術性能指標 TPM

TPM は、プロジェクト進行中に継続的に監視する技術的指標であり、システム完成前の段階で性能低下や設計上の問題を早期に発見するために用いる。本プロジェクトで監視する TPM を表??に示す。

表 1.3 MOE と MOP の対応

MOE	MOP	性能指標	目標値・評価基準
MOE1	MOP1	演奏タイミング誤差	各演奏側 Arduino 間の音符開始タイミングのズレが 30 ms 以内であること [?]
MOE1	MOP2	同期信号の受信成功率	指揮側から送信した赤外線信号を演奏側が正しく受信できる割合が 95% 以上であること
MOE2	MOP3	テンポ変更への追従時間	指揮側がテンポを変更してから演奏側全台が新テンポに追従するまでの時間が 1 拍以内であること
MOE3	MOP4	音色の再現性	倍音構造・ADSR の設定により, 対応する楽器の音色として聴衆の過半数に評価されること
MOE4	MOP5	歌詞・アニメーション同期遅延	演奏ログと LCD 表示切替タイミングを比較し, その差が 100 ms 以内であることを確認する

表 1.4 システムテストで評価する MOP

MOP	性能指標	テスト方法
MOP1	演奏タイミング誤差	「かえるのうた」を輪唱形式で通し演奏し, 各演奏側 Arduino の音符開始タイミングをオシロスコープまたは DAW で計測し, ズレが 30 ms 以内であることを確認する
MOP2	テンポ変更への追従時間	演奏中に指揮側のテンポを変更し, 全演奏側が 1 拍以内に追従することをシリアルモニタで確認する
MOP3	音色の再現性	Google フォームを用いてアンケートを作成し, 学科内で調査を行い, 作成した音が対応する楽器の音色として聴衆の過半数の評価を得る

表 1.5 結合テストで評価する MOP

MOP	性能指標	テスト方法
MOP1	演奏タイミング誤差	指揮側 Arduino から同期信号を送信し、複数の演奏側 Arduino が同時に音符開始信号を出力する時刻差をシリアルログで計測し、30 ms 以内であることを確認する
MOP2	同期信号の受信成功率	一定間隔で同期信号を 100 回送信し、演奏側 Arduino が 95 回以上正しく受信できることを確認する
MOP3	テンポ変更への追従時間	指揮側 Arduino の送信周期を変更し、演奏側 Arduino が 1 拍以内に新しい周期へ追従することをシリアルモニタで確認する
MOP5	歌詞・アニメーション同期遅延	音符開始信号と LCD 表示更新時刻をシリアルログで比較し、表示遅延が 100 ms 以内であることを確認する

表 1.6 単体テストで評価する MOP

MOP	性能指標	テスト方法
MOP2	同期信号の受信成功率	赤外線送受信モジュール間で同期信号を送信し, 100 回中 95 回以上正しく受信できることを確認する
MOP3	テンポ変更への追従時間	指揮側 Arduino の送信周期を変化させ, 演奏側 Arduino が周期変化を正しく認識することを確認する
MOP4	音色の再現性	Processing 単体で各音色を再生し, 倍音構造および ADSR パラメータ変更によって音色変化が生じることを確認する
MOP5	歌詞・アニメーション同期遅延	LCD モジュール単体で歌詞表示およびアニメーションを動作させ, 表示更新が正常に行われることを確認する

表 1.7 監視する TPM

TPM	監視項目	監視内容
TPM1	赤外線通信遅延	同期信号送信から受信までの時間を定期的に計測し、遅延増加が発生していないか確認する
TPM2	同期信号受信率	赤外線信号の受信成功率を定期的に測定し、受信率低下が発生していないか確認する
TPM3	Processing 音声再生遅延	Arduino からの信号受信後、Processing が音を再生するまでの時間を確認する
TPM4	LCD 表示遅延	音符開始信号から LCD 表示更新までの時間を定期的に計測する
TPM5	CPU 負荷・メモリ使用量	Arduino のメモリ使用量や処理負荷を確認し、動作不安定化が発生していないか監視する

1.4.7 プロジェクト中に監視する指標 TPM

プロジェクト期間中は、表??に示した TPM を定期的に測定し、システム性能が目標値から外れていないかを確認する。特に赤外線通信遅延および同期信号受信率については、複数台接続時に性能低下が発生する可能性があるため、結合の各段階で重点的に監視する。

また、音声再生遅延や LCD 表示遅延は、演奏の違和感に繋がるため、演奏テストを行うたびにログを取得し、遅延増加が発生していないか確認する。TPM が目標値を満たさない場合には、赤外線送信周期の見直しや同期アルゴリズムの修正等を実施し、性能改善を行う。

1.5 資源・調達管理

本プロジェクトで使用する資源と、その調達方法を表??に示す。このうち、Arduino 一台と赤外線 LED、押しボタンは指揮側で使用し、Arduino 4 台と赤外線受

信モジュール, LCD キャラクターディスプレイモジュール, USB ケーブル, PC は演奏側で使用する. また, ジャンパ線や抵抗ブレッドボードは双方で用いる.

表 1.8 使用資源と調達方法

使用資源	必要数	調達方法
Arduino 本体	5 台	各自持参
押しボタン	2 個	実験室より貸し出し
赤外線 LED (IR LED)	2 個	実験室より貸し出し
赤外線受信モジュール (IR センサ)	8 個	実験室より貸し出し
LCD キャラクターディスプレイモジュール	4 個	実験室より貸し出し
ブレッドボード	5 枚	各自持参
ジャンパ線	適宜	各自持参
USB ケーブル (Arduino-PC 接続用)	5 本	各自持参
抵抗 (IR LED 電流制限用)	適宜	各自持参
PC (Processing 実行用)	4 台	各自持参

1.6 リスク管理

本プロジェクト中に発生しうるリスクとして, 赤外線信号の誤検知・受信失敗や, 音声のずれの発生などがある. これらの対策として, 信号の誤検知や受信失敗が発生した場合には, 実験環境におけるセンサの閾値や, センサの向き変更, また外光の遮断などを試み原因の特定と通信機能の見直しを行う.

また, 音声のずれが発生した場合は, その原因が演奏側のメインループにあるのか, Processing との通信にあるのかを見極め, 原因に応じて LCD の表示内容の簡略化や, 送受信データ量の調整, データフォーマットの見直しを行う.

第 2 章 システムの基本設計

本システムは、指揮側である親機の Arduino 1 台から赤外線によって同期信号を送信し、演奏側である子機の Arduino 4 台がそれらを受信して、あらかじめ保存された楽譜データを基に Processing 上で楽器音を再生する、Arduino を用いたオーケストラシステムである。演奏楽曲は「かえるのうた」とし、輪唱形式で演奏する。また、子機の Arduino に接続した LCD キャラクターディスプレイにはカエルのドット絵アニメーション（口パク）と歌詞を表示する。

2.1 機能一覧

本システムの FBS に基づく機能一覧を以下に示す。

- ・同期制御機能
 - 赤外線による同期信号の送信（親機 Arduino）
 - 赤外線による同期信号の受信（子機 Arduino）
 - 赤外線の点滅テンポは楽曲中の最も早い拍に合わせる
- ・楽曲制御機能
 - 音符情報の生成（楽譜配列からの音高・強さ・長さ取得）
 - 演奏タイミング制御（同期信号に基づく発音タイミング管理）
 - 音声出力（Processing による楽器音再生）
- ・LCD 表示機能
 - かえるのドット絵アニメーション（口パク）表示
 - 演奏に合わせた歌詞スクロール表示

2.2 システム構成図

本システムは、親機 Arduino（1 台）、子機 Arduino（4 台）、および Processing を実行する PC により構成される。

2.2.1 ハードウェア構成

- ・ 親機 (指揮) Arduino (1 台)
 - 赤外線 LED (マスクデータ斉送信用) (1 台)
 - ボタン (演奏開始用)
 - ボタン (テンポ可変用)
 - ボタン (リセット用)
 - 楽器選択用ボタン (各楽器の開始予約用)
- “ “
- ・ 子機 (演奏) Arduino (4 台: ピアノ・フルート・トランペット・ドラム)
 - 赤外線受光センサ (同期マスク受信) (各 1 台)
 - LCD キャラクターディスプレイ (アニメーション・歌詞表示) (必要な子機に接続)
 - シリアル通信 (Processing 接続)
- ・ PC (Processing 実行環境)
 - 楽器音生成・スピーカ出力

“ “

2.2.2 ソフトウェア構成

- ・ 親機 (指揮側) Arduino プログラム
 - 演奏状態管理プログラム (OFF / QUEUED / ACTIVE)
 - 小節頭開始予約制御プログラム
 - マスク生成・赤外線送信プログラム
 - テンポ管理プログラム
- “ “
- ・ 子機 (演奏側) Arduino プログラム
 - 親機との同期プロトコル (マスク受信・発音判定)
 - 譜面進行プログラム
 - Processing との通信関数
 - LCD 表示関数 (アニメーション・歌詞)
- ・ Processing プログラム

- シリアル通信受信
- 音声再生関数（楽器音再生）
- 音色・エフェクト付与関数

““

システム全体の処理の流れを以下に示す。

親機 Arduino ↓ 赤外線マスク送信 子機 Arduino (×4 台) ↓ シリアル通信
Processing ↓ スピーカ出力

また、子機 Arduino は LCD へも並行して出力する。

子機 Arduino ↓ LCD キャラクターディスプレイ（アニメーション・歌詞）

2.3 操作インターフェース設計

本システムは主に自動演奏を目的とするが、演奏者が任意のタイミングで楽器の演奏開始を予約できるよう、親機側に複数の入力スイッチを設ける。操作インターフェースは以下のように構成する。

2.3.1 親機 Arduino

- ・演奏開始スイッチ
- ・テンポ制御スイッチ
- ・リセットスイッチ
- ・楽器選択スイッチ（ピアノ・フルート・トランペット・ドラム）
- ・赤外線 LED の動作確認

各楽器選択スイッチは、押した瞬間にただちに演奏を開始するのではなく、その楽器を開始予約状態（QUEUED）に設定するために用いる。予約された楽器は次の小節頭で演奏中状態（ACTIVE）へ遷移し、その時点から演奏を開始する。この方式により、演奏者は任意のタイミングで参加を指示できる一方で、実際の開始タイミングは小節先頭に揃えられるため、輪唱の同期精度を維持できる。

2.3.2 子機 Arduino

- ・赤外線受信状態確認
- ・シリアル通信状態表示

- ・ LCD によるアニメーション・歌詞表示

2.3.3 Processing 側

- ・ 楽器音再生
- ・ 受信データ確認
- ・ デバッグ用コンソール表示

Processing 側では受信した整数値に応じてあらかじめ設定された楽器音（ピアノ・フルート・トランペット・ドラム）を再生する。

2.4 状態遷移図

本システムの演奏側 Arduino の状態遷移を以下に示す。

1. 待機状態
 - ・ 赤外線同期マスク待機
2. 同期信号受信状態
 - ・ マスクデータ受信
 - ・ 自身の担当ビット確認
3. 開始判定状態
 - ・ 前回状態との比較
 - ・ OFF から ACTIVE へ遷移した場合は譜面位置を先頭に初期化
4. 演奏状態
 - ・ 譜面配列を 1 コマ進める
 - ・ Processing に演奏データ送信
 - ・ 楽器音再生
 - ・ LCD アニメーション・歌詞更新
5. 待機または継続状態
 - ・ 自身のビットが無効なら待機へ戻る
 - ・ 有効なら次の同期信号で継続演奏する

待機状態 ↓ 同期信号受信 ↓ 開始判定 ↓ 演奏状態 (Processing 送信 + LCD 更新) ↓ 次回同期信号待機

2.5 技術性能指標 (TPM)

本システムでは、以下の技術性能指標 TPM を管理対象とする (第 1 章 V&V 計画参照).

- ・ 赤外線受信成功率 (目標: 95% 以上)
- ・ Arduino 間の同期誤差 (目標: 50ms 以下)
- ・ 通信遅延 (目標: 50ms 以下)
- ・ テンポ追従時間 (目標: 3 秒以内)
- ・ 長時間動作安定性 (目標: エラーなし)
- ・ 同期可能 Arduino 台数 (目標: 4 台以上)

2.6 テスト計画

本プロジェクトでは以下のテストを実施する.

2.6.1 単体テスト

- ・ 赤外線信号送受信テスト
- ・ 楽譜配列参照テスト
- ・ シリアル通信テスト
- ・ 音源再生テスト
- ・ LCD 表示テスト (アニメーション・歌詞)

2.6.2 結合テスト

- ・ 親機 Arduino と子機 Arduino 間の同期確認
- ・ Arduino と Processing 間の接続確認
- ・ 複数楽器間のテンポ同期確認 (輪唱・同時演奏)

2.6.3 システムテスト

- ・「かえるのうた」の通し演奏
- ・長時間演奏時の安定性確認（5 分以上）
- ・複数端末同時演奏確認（4 台の演奏側 Arduino）

第 3 章 システムの詳細設計

設計書は作成する成果物によって変化します。適宜修正して利用してください。

3.1 部品一覧

本システムでは、製品分解構造（PBS: Product Breakdown Structure）の考え方をを用いて使用部品を整理する。PBS とは、システムを構成する製品要素を役割ごとに分解する手法である。本システムでは、物理的構成要素に基づき、PBS の要素を「Arduino 親機」と「Arduino 子機」の 2 つに分ける。なお、Processing は音生成を担うが物理部品ではないため、本節の PBS 要素には含めない。Arduino 親機は演奏全体の進行管理と同期情報の送信を担い、Arduino 子機は同期情報の受信と各楽器パートの制御を担う。表??に、本システムの部品一覧を示す。

表 3.1 PBS に基づく部品一覧

PBS 要素	部品名	数量
Arduino 親機	Arduino Uno R4	1
Arduino 親機	赤外線 LED	4
Arduino 親機	タクトスイッチ	2
Arduino 親機	抵抗器	
Arduino 親機	ジャンパワイヤ	
Arduino 子機	Arduino Uno R4	4
Arduino 子機	赤外線受光機 OSRB38C9AA	4
Arduino 子機	抵抗器	
Arduino 子機	ジャンパワイヤ	

3.2 ハードウェア設計

システムは親機 Arduino1 台, 子機 Arduino4 台, Processing を動作させる PC から構成される. 親機は楽器ごとの状態を管理し, 演奏中の楽器集合を表す mask データを赤外線 LED から全子機へ一斉送信する. 子機は受信した mask のうち自身の担当ビットを参照して演奏可否を判定し, 演奏対象である場合のみ内部に保持した譜面配列を進めて Processing へ音番号を送信する. また, 親機では各楽器に対して OFF, QUEUED, ACTIVE の 3 状態を管理し, 演奏者が任意のタイミングで入力した楽器開始要求をいったん予約として保持し, 小節頭で演奏開始させる.

3.2.1 Arduino 親機の構成

Arduino 親機は, システム全体の進行管理を担う構成要素である. 親機には Arduino Uno R4, 赤外線 LED, タクトスイッチを接続する. Arduino Uno R4 はテンポ管理, 現在の小節位置管理, 各楽器の状態管理を行い, 演奏中の楽器集合を表す mask データを赤外線 LED を通して子機へ一斉送信する. タクトスイッチは演奏開始, テンポ変更, リセット, および各楽器の開始予約入力に用いる. また, 赤外線 LED の駆動には電流制限のための抵抗器を用いる.

3.2.2 Arduino 子機の構成

Arduino 子機は, 各楽器パートを担当する構成要素である. 各子機には Arduino Uno R4 と赤外線受光機 OSRB38C9AA を接続する. 子機は赤外線受光機を用いて親機から送信された mask データを受信し, 自身の担当楽器に対応するビットが有効かどうかを判定する. ビットが有効である場合にのみ, 自身の譜面配列を 1 コマ進め, 対応する音番号を Processing に送信する. また, 前回無効で今回有効になった場合は, 譜面位置を先頭に戻して演奏開始を揃える. 本システムでは, ピアノ, フルート, トランペット, ドラムの 4 台の子機を用いる.

3.3 ソフトウェア設計

本システムのソフトウェアは, 親機 Arduino 用プログラム, 子機 Arduino 用プログラム, および音生成を担う Processing 用プログラムから構成される. 親機 Arduino

は曲の進行管理と同期情報の送信を担当し、子機 Arduino は同期情報の受信と譜面参照を担当する。Processing は各子機から送信された音番号に基づいて楽器音を生成し、再生する。本節では、これら 3 つのソフトウェアの役割と処理内容について述べる。

3.3.1 親機 Arduino のソフトウェア設計

親機 Arduino は、システム全体の進行制御を担当する。親機では、各楽器について待機状態 (OFF)、開始予約状態 (QUEUED)、演奏中状態 (ACTIVE) の 3 状態を管理する。演奏者が楽器選択ボタンを押すと、対応する楽器は QUEUED に設定され、次の小節頭で ACTIVE に遷移する。親機はテンポに応じて小節位置を更新し、小節頭に到達したときに予約済み楽器を演奏開始させる。その後、ACTIVE 状態にある楽器集合を表す mask を生成し、赤外線で子機へ送信する。親機で扱う主な処理は、テンポ管理、小節位置管理、QUEUED から ACTIVE への遷移処理、mask 生成処理、赤外線送信処理である。

3.3.2 子機 Arduino のソフトウェア設計

子機 Arduino は、各楽器パートの制御を担当する。各子機には担当楽器ごとの譜面配列を保持させる。子機は親機から赤外線で送信された mask を受信し、その mask のうち自身の担当楽器に対応するビットを確認する。自身のビットが有効な場合は譜面配列 myScore[] を 1 コマ進め、対応する音番号を Processing に送信する。ただし、前回は無効で今回初めて有効になった場合は、譜面位置を先頭に戻してから演奏を開始する。休符に対応する場合は発音を行わない。この構成により、各子機は親機の一斉送信を共通に受け取りながら、自分が演奏対象となったときのみ同期して参加できる。

3.3.3 ファイル構成

本システムのソフトウェアは、Arduino 側プログラムと Processing 側プログラムから構成される。Arduino 側では、親機用プログラム 1 つと、各楽器を担当する子機用プログラム 4 つを用意する。Processing 側では、各楽器に対応した音色生成プログラムを個別に用意し、Arduino から送信された音番号に基づいて音を再生する。各ファイルの役割を以下に示す。

3.3.4 ファイル構成

本システムのソフトウェアは、Arduino 側プログラムと Processing 側プログラムから構成される。Arduino 側では、親機用プログラム 1 つと、各楽器を担当する子機用プログラム 4 つを用意する。Processing 側では、各楽器に対応した音色生成プログラムを個別に用意し、Arduino から送信された音番号に基づいて音を再生する。各ファイルの役割を以下に示す。

- ・ master.ino：親機 Arduino 用プログラム。各楽器の OFF / QUEUED / ACTIVE 状態管理、小節頭での開始予約反映、mask の生成、赤外線による同期情報送信を行う。
- ・ child_{piano}.ino：ピアノ担当子機用プログラム。受信したmaskを基に発音可否を判定し、譜面配列を。
- ・ child_{trumpet}.ino：トランペット担当子機用プログラム。受信したmaskを基に発音可否を判定し、譜面配
- ・ piano.pde：ピアノ音色を生成し、再生するための Processing プログラム。
- ・ flute.pde：フルート音色を生成し、再生するための Processing プログラム。
- ・ trumpet.pde：トランペット音色を生成し、再生するための Processing プログラム。
- ・ drum.pde：ドラム音を生成し、再生するための Processing プログラム。

3.3.5 関数設計

本システムで使用する主要関数を、親機 Arduino、子機 Arduino、Processing ごとに示す。

3.3.5.1 親機 Arduino の関数

表 3.2 親機 Arduino の主要関数

関数名	内容
initMasterSystem()	親機の初期設定を行う。
updateStep()	曲位置 step を更新する。
updateActiveInstrument()	演奏対象の楽器を決定する。
encodeSyncData(int step, int activeInstrument)	step と楽器番号を送信用データにまとめる。
sendSyncData(uint16_t data)	同期データを赤外線で送信する。
handleTempoInput()	ボタン入力に応じてテンポや再生状態を変更する。

3.3.5.2 子機 Arduino の関数

表 3.3 子機 Arduino の主要関数

関数名	内容
initChildSystem()	子機の初期設定を行う.
receiveSyncData()	親機からの同期データを受信する.
decodeStep(uint16_t data)	受信データから step を取り出す.
decodeActiveInstrument(uint16_t data)	受信データから楽器番号を取り出す.
readScoreValue(int step)	譜面配列から音番号を取得する.
shouldPlayNote(int note, int activeInstrument)	発音すべきか判定する.
sendNoteToProcessing(int note)	音番号を Processing に送信する.

3.3.5.3 Processing の関数

表 3.4 Processing の主要関数

関数名	内容
initAudioSystem()	音響システムを初期化する.
serialEvent(Serial p)	シリアルデータを受信して解析する.
parseNote(String msg)	受信データから音番号を取り出す.
dispatchInstrumentSound(int instrument, int note)	楽器に応じた再生関数を呼び出す.
playPianoSound(int note)	ピアノ音を再生する.
playFluteSound(int note)	フルート音を再生する.
playTrumpetSound(int note)	トランペット音を再生する.
playDrumSound(int note)	ドラム音を再生する.

3.4 処理フローの設計

本節では、親機 Arduino、子機 Arduino、および Processing の 3 つのコンポーネントについて、それぞれの処理フローをフローチャートで示す。各コンポーネントは独立したプログラムとして動作し、赤外線通信とシリアル通信を介して連携する。

3.4.1 親機 Arduino の処理フロー (master.ino)

親機 Arduino は、曲位置 `step` と演奏対象楽器 `activeInstrument` を管理し、一定のテンポ周期で同期データを赤外線送信し続けるメインループ構造をとる。ボタン入力により演奏の開始・停止・テンポ変更が可能であり、`isPlaying` フラグが `true` の場合のみ `step` の更新と送信を行う。

3.4.2 子機 Arduino の処理フロー (child_xxx.ino)

子機 Arduino は、親機から送信される同期データを受信し、自身の担当楽器が演奏対象であれば Processing に音番号を送信する。受信ごとに `step` と `activeInstrument` を復号し、譜面配列 `myScore[step]` を参照したうえで `shouldPlayNote()` で発音可否を判断する。休符（音番号 0）の場合や担当外の楽器番号の場合は送信を行わない。

3.4.3 Processing の処理フロー (xxx.pde)

Processing は、シリアル通信の割り込みによって各子機から音番号を受信し、担当楽器に応じた音色再生関数を呼び出す。`setup()` で音響システムを初期化した後は、`serialEvent()` が受信ごとに自動で呼ばれる割り込み駆動の構造である。受信データを `parseNote()` で解析し、`dispatchInstrumentSound()` で楽器番号に応じた再生関数へ振り分ける。

3.4.4 3つのコンポーネント間のデータフロー

図 ?? に、親機・子機・Processing 間のデータの流れを示す。親機が赤外線同期データ (`step`, `activeInstrument`) を全子機へブロードキャストし、各子機が自身の判断でシリアル通信により Processing へ音番号を送信する構成である。

3.5 テストの設計

本システムが要件を満たして正しく動作することを確認するために、単体テスト、結合テスト、システムテストを実施する。テスト計画ではテスト項目の概要のみを示したが、本節では各テストについて、確認対象、テスト方法、テストデータ、期待結果を具体的に記述する。各テストでは、シリアルモニタ出力、実機の動作確認、およ

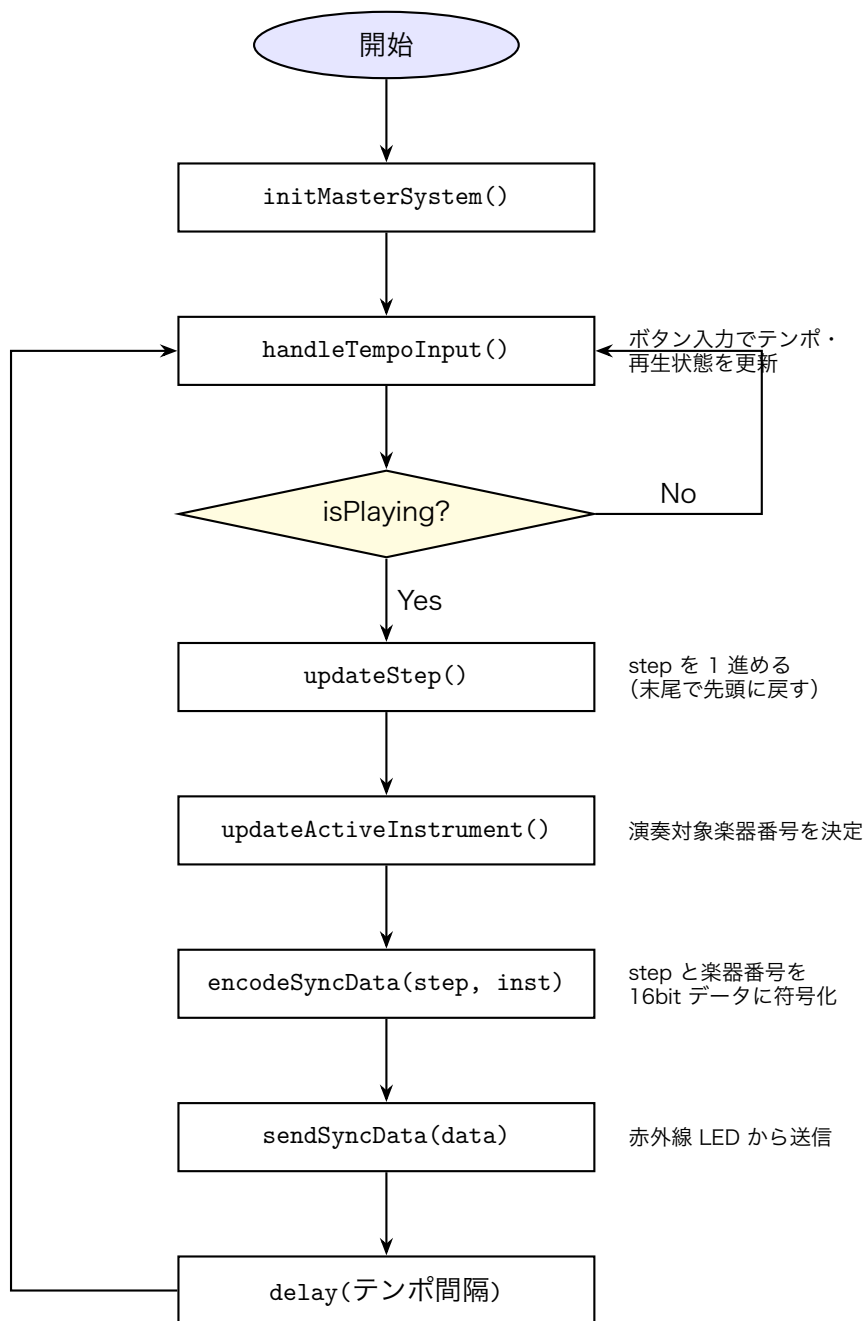


図 3.1 親機 Arduino の処理フロー

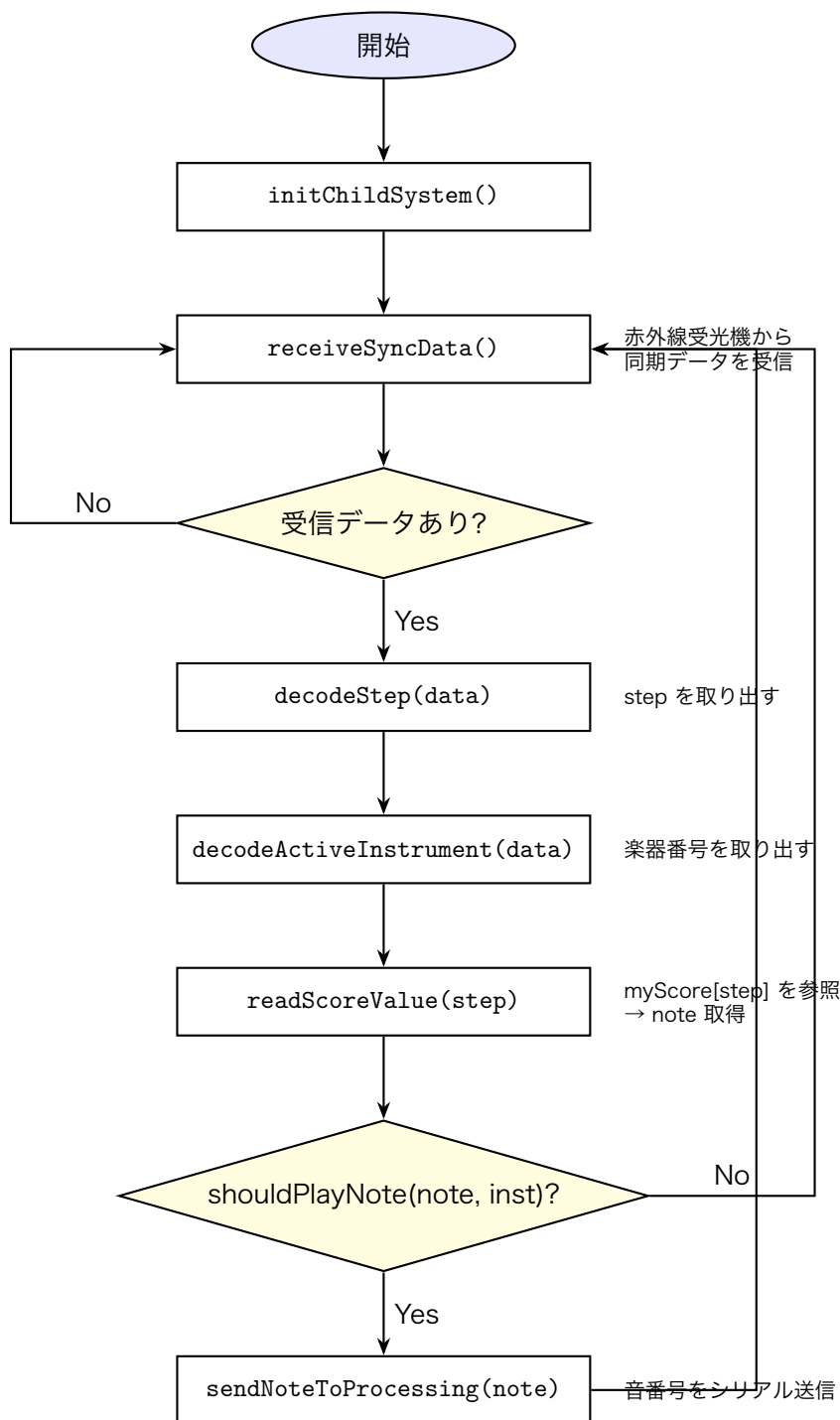


図 3.2 子機 Arduino の処理フロー（各楽器共通）

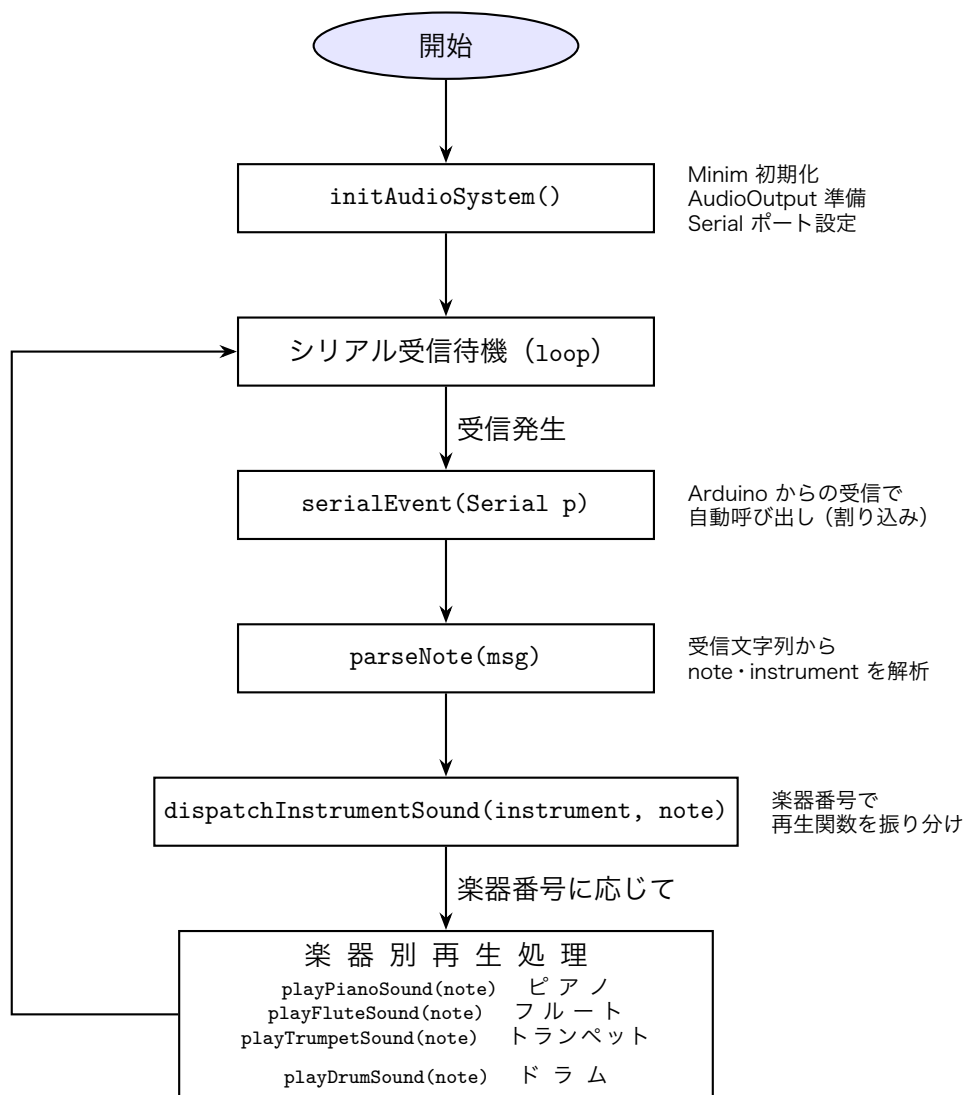


図 3.3 Processing の処理フロー

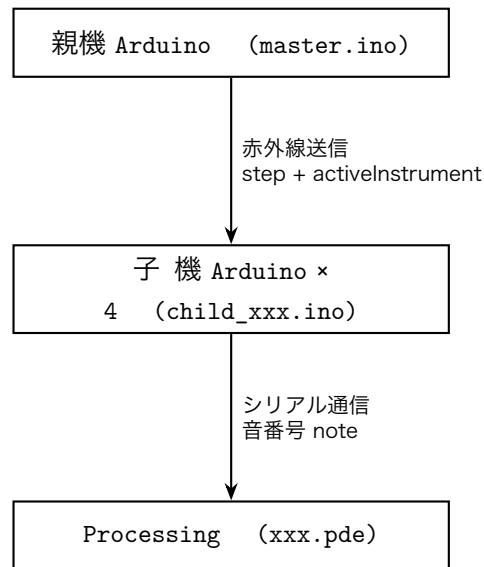


図 3.4 コンポーネント間のデータフロー

び実際の音の再生結果を用いて評価を行う。

3.5.1 単体テスト

各成果物を単独で動作させ、個別の機能が正しく動作することを確認する。親機 Arduino の単体テストでは、テンポ管理、小節位置更新、各楽器の OFF / QUEUED / ACTIVE 状態遷移、および ACTIVE な楽器集合から正しく mask を生成できることを確認する。特に、楽器選択入力によって対象楽器が QUEUED に設定され、小節頭で ACTIVE に遷移することを確認する。子機 Arduino の単体テストでは、受信した mask に対して自身の担当ビットを正しく判定できること、前回無効かつ今回有効であった場合に譜面位置を先頭に戻せること、その後は有効な受信ごとに譜面配列を 1 コマずつ進められることを確認する。Processing の単体テストでは、Arduino から受信した音番号に応じて対応する楽器音を再生できること、休符に対して不要な発音を行わないことを確認する。

3.5.2 結合テスト

3.5.2.1 親機 Arduino と子機 Arduino 間の同期確認

親機と子機を接続し、親機から送信した step と activeInstrument に子機が追従できるかを確認する。テストでは、親機を一定テンポで動作させ、子機側で受信した

step をシリアルモニタに表示する。期待結果は、複数回の送受信において子機が親機と同じ曲位置を共有できることである。

3.5.2.2 Arduino と Processing 間の接続確認

子機 Arduino と Processing を接続し、子機から送信した音番号に対して Processing が正しく応答するかを確認する。テストでは、子機で特定の音番号を送信し、Processing で対応する音が鳴るかを確認する。期待結果は、送信した音番号に対応した音が遅延なく再生されることである。

3.5.2.3 複数楽器間のテンポ同期確認

複数の子機を同時に動作させたときに、テンポがずれずに同期演奏できるかを確認する。テストでは、親機 1 台と子機 4 台を接続し、同じ課題曲を演奏させる。期待結果は、各子機が同一の step を共有し、楽器切替や同時発音のタイミングが大きくずれないことである。

3.5.3 システムテスト

3.5.3.1 「かえるのうた」の通し演奏

システム全体として課題曲を最初から最後まで演奏できるかを確認する。テストでは、親機、子機、Processing をすべて接続した状態で「かえるのうた」を通して演奏する。期待結果は、演奏が途中で停止せず、曲全体を通して同期が維持されることである。

3.5.3.2 長時間演奏時の安定性確認

長時間動作させた場合でも、通信や演奏が安定して継続するかを確認する。テストでは、少なくとも 5 分以上連続して演奏を行う。期待結果は、赤外線通信の途切れや誤動作が発生せず、演奏が継続できることである。

3.5.3.3 複数端末同時演奏確認

4 台の子機 Arduino を同時に動作させた場合でも、すべての端末が正常に演奏へ参加できるかを確認する。テストでは、ピアノ、フルート、トランペット、ドラムの各子機を同時に接続し、親機から同期情報を送信する。期待結果は、各子機が自分の担

当タイミングで正しく音番号を送信し，合奏として成立することである.